EE5203 L12 Lab Guide

Project clinic — the readiness review - Basri Erdogan

Mission: Bring the whole system to the bench and prove it holds together — a resource map with no conflicts, a timing budget that fits, one fault found by bisection rather than guesswork, and a requirement list with a real test beside each line.

This is a **working session**. There is no new peripheral to configure. Bring the problem you are most afraid of, not the part you are proudest of.

Prepare before class

- ☐ Re-read the theory slides, especially the timing budget and bisection.
- ☐ Fill in your **resource map** and **timing budget** (templates below).
- ☐ Write your requirement list — one testable sentence per requirement.
- ☐ Commit and **tag** the version you are bringing. You will be asked to return to it.
- ☐ Bring the whole system: board, wiring, sensors, actuators, power.

Learning objectives

By the end of the session, you can:

1. present a conflict-free resource map for your own project;
2. defend a timing budget with arithmetic;
3. isolate a fault by bisection and record the steps;
4. state each requirement with the test that demonstrates it.

Deliverable 1 - The resource map

Every peripheral, pin, and timer your project uses, with exactly one owner each.

Resource	Pin(s)	Owner / purpose	Conflict?

Check specifically: two functions on one pin; two frequencies on one timer; `HAL_Delay` reachable from an interrupt; two ADC channels assumed simultaneous.

Deliverable 2 - The timing budget

Activity	Cost per occurrence	Rate	Load %
Total			

Costs you can measure rather than guess: toggle a spare GPIO around the section and capture it — with a scope, or with the input-capture channel you built in L08. Anything above roughly 50 % total needs a plan; anything above 100 % is a design error, not a bug.

Checkpoint A - The system runs

Evidence: ten minutes of continuous operation with telemetry flowing and no resets. Show the timestamp column advancing without gaps or restarts.

A reset mid-run is not a failed checkpoint — it is a finding. Record when it happened and what was running, and put it on the known-issues list.

Checkpoint B - Map and budget

Evidence: both tables complete, with no unresolved conflicts and a stated total load. You must be able to say which single activity dominates and what you would cut first.

Checkpoint C - One fault, bisected

Pick a real fault — current, or one you fixed this week and can reproduce.

Evidence: a written bisection log:

Step	What was disabled	Fault still present?	Conclusion
1			
2			

The log is the deliverable, not the fix. A team that fixed something by changing three things at once has not met this checkpoint.

Checkpoint D - Requirements and tests

Evidence: a table with a testable requirement per row, the test procedure, and the result you actually observed.

#	Requirement	Test	Result
1			

Plus an honest **known-issues list**. Every real system has one. An empty list at this stage reads as untested, not as finished — and the L13 defence will find what you did not.

The clinic itself

Ten minutes per team with the instructor:

1. show the system in its current state (2 min);
2. present the map and budget (3 min);
3. name your **top three risks** and the plan for each (3 min);
4. leave with a written list of what must be true by L13 (2 min).

Acceptance checklist

- ☐ Resource map complete, one owner per resource, no unresolved conflicts.
- ☐ Timing budget complete, with a stated total and the dominant item named.
- ☐ Ten-minute run with telemetry, no unexplained resets.
- ☐ Bisection log with at least three steps and a conclusion.
- ☐ Requirements table with tests and observed results.
- ☐ Known-issues list, honest and specific.
- ☐ Version committed and tagged.

Individual oral check

Each member answers about **their own** contribution:

- Which resources do you own, and what would conflict with them?
- What does your part cost in the timing budget? Show the arithmetic.
- Describe a fault you found and the steps you took to isolate it.
- Which requirement is your part responsible for, and how is it tested?
- What is the biggest risk to your demonstration, and what is the fallback?

Submit

Submit the resource map, timing budget, bisection log, requirements table, and known-issues list as one document, plus your tagged commit hash.

If you are stuck

Nothing works together but each part works alone: start with the resource map — the conflict is usually visible on paper. **Intermittent faults:** add counters (`ore_count`, `nack_count`, missed deadlines) and print them once a second; intermittent becomes measurable. **Resets under load:** check the current budget and the shared ground before suspecting software. **"It worked yesterday":** `git bisect` between the last known-good commit and now.